

Job Market Analysis

PROJECT TITLE: JOB MARKET ANALYSIS

NAME: A. PRANUSHA

BATCH: DA-BN001

Objectives:

Job Market Analysis using SQL, Python, and Power BI

The main objectives of this project are:

1. **Understand the global job market trends** by analyzing a dataset containing job postings from multiple industries, countries, and companies.
2. **Identify high-paying roles, skill demand, and location advantages** to provide career-oriented insights.
3. **Explore the relationship between job titles, experience levels, education, and salary** using descriptive and visual analytics.
4. **Clean and transform job data** for better usability and structure using SQL and Python (Pandas).
5. **Build visual dashboards** using Python (Seaborn/Matplotlib) or Power BI to clearly communicate insights for stakeholders.

Data Collection:([Kaggle](#))

- Source: Kaggle (public dataset)
- File Format: CSV (Comma-Separated Values)
- Dataset Size: 15,000 rows × 20 columns
- Key Fields Include:
 - job_title: Title of the job position
 - company_name: Employer or organization offering the job
 - location and country: Job location
 - employment_type: Nature of job (Full-time, Part-time, etc.)

- `experience_level`: Entry-level, Mid-level, Senior, etc.
- `education_required`: Minimum education qualification
- `salary_usd` and `salary_local`: Salary offered in USD and local currency
- `posting_date` and `application_deadline`: Dates related to job postings
- `remote_ratio`: Percentage of remote work allowed
- `required_skills`: Skills needed for the role
- `job_description_length`: Text length of the job description

Tools & Technologies Used:

- 1) **MySQL**: Data extraction, initial filtering, grouping, and aggregation using SQL queries
- 2) **Python (Pandas)**: Data cleaning, preprocessing, and transformation
- 3) **Python (Matplotlib & Seaborn)**: Exploratory data analysis and visualization
- 4) **Jupyter Notebook**: Interactive development and analysis environment for Python
- 5) **Power BI**: dashboarding and interactive visual reports

Data Cleaning — Python (Pandas):

Missing Values Handling

- Checked for null values in each column.
- Filled or dropped missing values based on the column type and business logic.
- Example: For `salary_usd`, if any missing values were found, they were filled using the median salary.

Incorrect Data Types

- Ensured numerical columns like `salary_usd`, `salary_local`, and `remote_ratio` were correctly converted to float or int.
- Converted date columns (`posting_date`, `application_deadline`) to datetime format for time-based analysis.

Zero Values

- Replaced invalid placeholders like "0" in columns such as `required_skills` with empty or NaN.

Standardization

- Cleaned text data by stripping whitespaces and converting to lowercase.

- Unified inconsistent values in categorical columns like `employment_type` and `experience_level`.

Descriptive Analysis:

--> Questions to Explore:

1. What are the top 10 highest-paying job titles?
2. Which countries have the highest average salary?
3. How many jobs are listed by each company?
4. How many jobs are posted per industry?
5. How many remote vs on-site jobs exist?
6. Which experience levels have the most job postings?

Stage 2: Data Cleaning — Python (Pandas)

Goal: Prepare clean and reliable data for analysis

What You Do:

Handle missing values

Fix inconsistent values (e.g., Bachelor vs Bachelors)

Convert columns to correct data types

Extract new columns (e.g., application window)

Normalize or split text fields

Cleaning Tasks/Questions:

1. Are there missing or null values in any columns?
2. Are date columns in the correct format?
3. Are all salary values numerical?
4. Can I create a new column: `application_duration = deadline - posting_date`?
5. How many unique values are in `education_required`?
6. Can I extract a skill count from `required_skills`?

Stage 3: Data Transformation — SQL

Goal: Prepare summarized, filtered, or derived data in SQL for analysis or visualization

--> What You Do:

Create views or temp tables with calculated fields

Aggregate data (grouped summaries)

Categorize experience or salary

Apply CASE statements for grouping

Transformation Questions:

1. What is the average salary per education level?
2. Can we categorize experience into "Junior", "Mid", "Senior", "Expert"?
3. What's the distribution of remote types (0, 50, 100) as labels?
4. Which industries offer the longest application windows?
5. What are the top 5 most common required skills?

Stage 4: Data Visualization — Power BI or Python

Goal: Present key insights with charts and dashboards

--> What You Do:

Create visual dashboards (Power BI) or

Plot charts using Seaborn/Matplotlib in Python

Visualization Questions & Ideas:

Bar chart: Top 10 highest-paying job titles

1. Map or bar chart: Salary by country
2. Bar/Box plot: Education level vs Salary
3. Pie chart: Remote work distribution
4. WordCloud: Most common required skills

5.Line plot: Job postings over time (posting_date)

6.Heatmap: Correlation between salary, benefits score, years of experience

Outcome of Cleaning

After cleaning:

- All salary columns are numeric and usable for aggregation.
- Nulls and invalid entries are handled.
- The dataset is consistent, structured, and ready for transformation and visualization.

Let me show with screenshots and with explanation

➤ Installing Required Libraries:

!pip install sqlalchemy pymysql

Explanation:

Before starting any data-related task, we need to make sure that all required Python packages are installed. This command installs two important libraries:

1. sqlalchemy:
 - SQLAlchemy is a powerful SQL toolkit and Object Relational Mapper (ORM) for Python.
 - It allows you to connect Python to a variety of SQL databases like MySQL, PostgreSQL, SQLite, etc.
 - It helps write Python code to interact with databases instead of writing raw SQL.
2. pymysql:
 - PyMySQL is a pure-Python MySQL client.
 - It is used by SQLAlchemy to connect with MySQL databases.
 - Without this, SQLAlchemy cannot talk to a MySQL database.

Why use !pip install?

- The ! symbol is used in Jupyter Notebooks to run shell commands.
- If you're working in Jupyter Notebook or Google Colab, use !pip install.

```
[2]: !pip install sqlalchemy pymysql

Requirement already satisfied: sqlalchemy in c:\users\user\anaconda3\lib\site-packages (2.0.34)
Requirement already satisfied: pymysql in c:\users\user\anaconda3\lib\site-packages (1.1.1)
Requirement already satisfied: typing-extensions>=4.6.0 in c:\users\user\anaconda3\lib\site-packages (from sqlalchemy) (4.11.0)
Requirement already satisfied: greenlet!=0.4.17 in c:\users\user\anaconda3\lib\site-packages (from sqlalchemy) (3.0.1)
```

It shows like this if it already downloaded if not it will download.

➤ Connect to MySQL and Read Data into Pandas:

➤ Syntax:

```

from sqlalchemy import create_engine
from urllib.parse import quote_plus # helps escape special chars
import pandas as pd
--sqlalchemy.create_engine builds a lightweight “engine” object that knows how to
talk to your database.
--quote_plus safely encodes special characters ( @, &, spaces ) that can break a
URL-style connection string.
--pandas gives us the convenient read_sql helper to pull query results straight into a
DataFrame

```

Credentials

```
username = 'root'
```

```
password = 'pruanu@&12345'
```

```
host = 'localhost'
```

```
database = 'job_market'
```

➤ **# URL-encode the password so @ & etc. don't break the URI**

```
encoded_pw = quote_plus(password)
```

- quote_plus converts special symbols into their percent-encoded equivalents (@ → %40, & → %26), ensuring the final connection URI remains valid.

➤ **# Build the SQLAlchemy engine string correctly**

```
engine = create_engine(
```

```
    f"mysql+pymysql://{username}:{encoded_pw}@{host}/{database}"
```

```
)
```

- Dialect + Driver – mysql+pymysql:// tells SQLAlchemy to speak MySQL using the *PyMySQL* driver.
- Credentials – we inject username and the encoded password.
- Hostname / DB – after @, specify the server (localhost) and default database (job_market).
The resulting engine can open pooled connections on demand; you don't need to engine.connect() manually for simple reads/writes.

➤ **# Load table into a DataFrame**

```
df = pd.read_sql("SELECT * FROM job_dataset", engine)
```

- Executes the SQL query (SELECT * FROM job_dataset) through the engine.
- Streams the result set into a pandas.DataFrame, automatically assigning column names and dtypes.

```
[3]: from sqlalchemy import create_engine
from urllib.parse import quote_plus # helps escape special chars
import pandas as pd

# Credentials
username = 'root'
password = 'pruanu@&12345'
host = 'localhost'
database = 'job_market'

# URL-encode the password so @ & etc. don't break the URI
encoded_pw = quote_plus(password)

# Build the SQLAlchemy engine string correctly
engine = create_engine(
    f"mysql+pymysql://{username}:{encoded_pw}@{host}/{database}"
)

# Load table into a DataFrame
df = pd.read_sql("SELECT * FROM job_dataset", engine)

print(df.head())
```

	job_id	job_title	salary_usd	salary_currency	\
0	AI00001	Data Scientist	219728	USD	
1	AI00002	Head of AI	230237	JPY	
2	AI00003	Data Engineer	128890	EUR	
3	AI00004	Computer Vision Engineer	96349	USD	

	job_id	job_title	salary_usd	salary_currency	\
0	AI00001	Data Scientist	219728	USD	
1	AI00002	Head of AI	230237	JPY	
2	AI00003	Data Engineer	128890	EUR	
3	AI00004	Computer Vision Engineer	96349	USD	
4	AI00005	Robotics Engineer	63065	EUR	

	salary_local	experience_level	employment_type	company_location	\
0	219728	EX	PT	Sweden	
1	25326070	EX	PT	Japan	
2	109557	EX	CT	Germany	
3	96349	MI	FL	Finland	
4	53605	EN	FT	France	

	company_size	employee_residence	remote_ratio	\
0	M	Sweden	0	
1	L	Japan	50	
2	S	Germany	100	
3	L	Finland	50	
4	S	France	100	

	required_skills	education_required	years_experience	\
0	Python, Computer Vision, R, Docker	Associate	13	
1	Kubernetes, MLOps, Tableau, Python	Bachelor	10	
2	Spark, Scala, Hadoop, PyTorch, GCP	Bachelor	12	
3	MLOps, Linux, Tableau, Python	PhD	2	
4	R, Scala, SQL, GCP, Python	Associate	0	

	industry	posting_date	application_deadline	job_description_length	\
--	----------	--------------	----------------------	------------------------	---

print(df.head())

Shows the first five rows—a quick sanity check that:

1. The connection worked.
2. Columns look as expected.
3. Row count is non-zero.

Print(tail()): It prints by default last 5 records

[6]: `df.tail()`

	job_id	job_title	salary_usd	salary_currency	salary_local	experience_level	employment_type	company_location	company_size	em
14995	AI14996	AI Product Manager	39171	USD	39171	MI	CT	China	M	
14996	AI14997	AI Product Manager	77555	EUR	65922	EN	CT	Netherlands	M	
14997	AI14998	Data Engineer	28380	USD	28380	EN	FL	India	M	

Data Cleaning:

info() gives information of number of features(columns), datatype and memory usage

df.info():

```
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15000 entries, 0 to 14999
Data columns (total 20 columns):
 #   Column                                Non-Null Count  Dtype
---  ---                                -
 0   job_id                               15000 non-null  object
 1   job_title                           15000 non-null  object
 2   salary_usd                          15000 non-null  int64
 3   salary_currency                     15000 non-null  object
 4   salary_local                        15000 non-null  int64
 5   experience_level                    15000 non-null  object
 6   employment_type                     15000 non-null  object
 7   company_location                    15000 non-null  object
 8   company_size                        15000 non-null  object
 9   employee_residence                  15000 non-null  object
10   remote_ratio                        15000 non-null  int64
11   required_skills                     15000 non-null  object
12   education_required                  15000 non-null  object
13   years_experience                    15000 non-null  int64
14   industry                            15000 non-null  object
15   posting_date                        15000 non-null  object
16   application_deadline                15000 non-null  object
17   job_description_length              15000 non-null  int64
18   benefits_score                      15000 non-null  float64
19   company_name                        15000 non-null  object
dtypes: float64(1), int64(5), object(14)
memory usage: 2.3+ MB
```

#describe shows statistics of data mean,mode,std,%,max,min,count

df.describe()

```
[8]: #describe shows statistics of data mean,mode,std,%,max,min,count
df.describe()
```

	salary_usd	salary_local	remote_ratio	years_experience	job_description_length	benefits_score
count	15000.000000	1.500000e+04	15000.000000	15000.000000	15000.000000	15000.000000
mean	121991.938267	8.292366e+05	50.196667	6.365667	1500.852600	7.499540
std	63968.361846	3.425325e+06	40.844084	5.598551	574.724647	1.444202
min	16621.000000	1.662100e+04	0.000000	0.000000	500.000000	5.000000
25%	74978.500000	7.383075e+04	0.000000	2.000000	998.750000	6.300000
50%	107261.500000	1.090355e+05	50.000000	5.000000	1512.000000	7.500000
75%	155752.250000	1.673278e+05	100.000000	10.000000	1994.000000	8.800000
max	410273.000000	3.368541e+07	100.000000	19.000000	2499.000000	10.000000

```
[9]: df.shape
```

```
[9]: (15000, 20)
```

df.shape: it gives total number of rows and columns as shown above.

➤ Identifying Missing Values:

➤ Syntax:

```
fnull=df.isnull()
print(fnull)
```

Explanation:

- df.isnull() checks each cell in the DataFrame and returns a boolean DataFrame of the same shape.
- If a value is NaN (null or missing), it returns True; otherwise, it returns False.

```
[10]: fnull=df.isnull()
print(fnull)
```

	job_id	job_title	salary_usd	salary_currency	salary_local	\
0	False	False	False	False	False	
1	False	False	False	False	False	
2	False	False	False	False	False	
3	False	False	False	False	False	
4	False	False	False	False	False	
...	...	...	...	...	...	
14995	False	False	False	False	False	
14996	False	False	False	False	False	
14997	False	False	False	False	False	
14998	False	False	False	False	False	
14999	False	False	False	False	False	

	experience_level	employment_type	company_location	company_size	\
0	False	False	False	False	
1	False	False	False	False	
2	False	False	False	False	
3	False	False	False	False	
4	False	False	False	False	
...	...	...	...	...	
14995	False	False	False	False	
14996	False	False	False	False	
14997	False	False	False	False	
14998	False	False	False	False	
14999	False	False	False	False	

Check for null/missing values:

df.isnull().sum():

It gives count of how many missing values are in each column

```
[11]: df.isnull().sum()

[11]: job_id          0
      job_title       0
      salary_usd      0
      salary_currency  0
      salary_local     0
      experience_level  0
      employment_type  0
      company_location  0
      company_size     0
      employee_residence 0
      remote_ratio     0
      required_skills   0
      education_required 0
      years_experience  0
      industry         0
      posting_date      0
      application_deadline 0
      job_description_length 0
      benefits_score    0
      company_name      0
      dtype: int64
```

print(df.duplicated()): It gives true if any duplicates exists if not it displays as a false and also dtype and length.

```
[12]: print(df.duplicated())

0      False
1      False
2      False
3      False
4      False
...
14995  False
14996  False
14997  False
14998  False
14999  False
Length: 15000, dtype: bool
```

print(df['benefits_score'].mean()): It gives mean of benefits_score.

```
[13]: print(df['benefits_score'].mean())

7.4995400000000005
```

Convert date columns:

Syntax:

```
df['posting_date'] = pd.to_datetime(df['posting_date'])
```

```
df['application_deadline'] = pd.to_datetime(df['application_deadline'])
```

Explanation:

- In many raw datasets, date columns are read as strings (object dtype) by default.
- `pd.to_datetime()` converts these string values into `datetime64[ns]` format, which makes it easier to:
 - Extract parts of dates (like year, month, weekday)
 - Calculate time differences (e.g., days until deadline)

By doing `df.info()` we can see the changes done before step that is converted datatype of object to `datetime64[ns]`.

```
[15]: df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15000 entries, 0 to 14999
Data columns (total 20 columns):
 #   Column                Non-Null Count  Dtype  
---  -
 0   job_id                15000 non-null  object  
 1   job_title             15000 non-null  object  
 2   salary_usd            15000 non-null  int64   
 3   salary_currency       15000 non-null  object  
 4   salary_local          15000 non-null  int64   
 5   experience_level      15000 non-null  object  
 6   employment_type       15000 non-null  object  
 7   company_location     15000 non-null  object  
 8   company_size          15000 non-null  object  
 9   employee_residence    15000 non-null  object  
10   remote_ratio          15000 non-null  int64   
11   required_skills       15000 non-null  object  
12   education_required    15000 non-null  object  
13   years_experience      15000 non-null  int64   
14   industry              15000 non-null  object  
15   posting_date          15000 non-null  datetime64[ns]
16   application_deadline  15000 non-null  datetime64[ns]
17   job_description_length 15000 non-null  int64   
18   benefits_score        15000 non-null  float64  
19   company_name          15000 non-null  object  
dtypes: datetime64[ns](2), float64(1), int64(5), object(12)
memory usage: 2.3+ MB
```

Created new columns:

Syntax:

```
df['application_duration'] = df['application_deadline'] - df['posting_date']
```

Explanation:

- This line creates a **new column** called `application_duration`.
- It subtracts `posting_date` from `application_deadline` to calculate **how many days were available for applying** to each job.
- Since both columns are in `datetime64[ns]` format, the result is a **timedelta object**, which shows the difference in days.

[20]: df

[20]:

	ars_experience	industry	posting_date	application_deadline	job_description_length	benefits_score	company_name	application_duration
13	Transportation	2024-09-23	2024-10-31	1132	6.6	TechCorp Inc	38 days	
10	Transportation	2024-07-26	2024-09-12	2299	8.5	Cloud AI Solutions	48 days	
12	Automotive	2025-01-19	2025-03-28	1329	5.5	Quantum Computing Inc	68 days	
2	Automotive	2024-07-20	2024-09-06	1132	6.8	Cognitive Computing	48 days	

It shows extra column which s created just now ie.s 21 columns and in above table it is the `application_duration`.

15000 rows × 21 columns



This column can be useful for:

- Analyzing how long companies keep job postings open
- Identifying short-duration or urgent job openings

Clean and Standardize Categorical Columns

Normalize text columns:

Syntax:

```
df['education_required'] = df['education_required'].str.title().str.strip()
```

```
df['experience_level'] = df['experience_level'].str.upper().str.strip()
```

▼ Normalize text columns

```
[15]: df['education_required'] = df['education_required'].str.title().str.strip()  
      df['experience_level'] = df['experience_level'].str.upper().str.strip()
```

Explanation:

When dealing with text data (like education or experience level), values often appear in inconsistent formats due to:

- Typing differences ("bachelor" vs "Bachelor " vs "BACHELOR")
- Extra spaces
- Lowercase/uppercase mismatches

We use string cleaning methods to make the data consistent:

`str.title()` – for `education_required`

- Converts strings to title case (first letter capitalized, rest lowercase).
- Example:
"bachelor's" → "Bachelor'S" → "Bachelor'S" (depending on punctuation)
" master " → "Master"

`str.upper()` – for `experience_level`

- Converts all text to uppercase.
- Example:
"entry" → "ENTRY"
"Senior " → "SENIOR"

`str.strip()` – removes leading and trailing spaces

- " Bachelor " → "Bachelor"

Why this matters:

- Makes filtering and grouping more accurate.
- Prevents duplicate categories like 'Senior', 'senior ', and 'SENIOR' being treated as different values.

Normalize text columns

```
[15]: df['education_required'] = df['education_required'].str.title().str.strip()  
df['experience_level'] = df['experience_level'].str.upper().str.strip()
```

Check unique values after cleaning:

Syntax:

```
print(df['education_required'].unique())
```

```
print(df['experience_level'].unique())
```

Check unique values after cleaning

```
[16]: print(df['education_required'].unique())  
print(df['experience_level'].unique())  
  
['Associate' 'Bachelor' 'Phd' 'Master']  
['EX' 'MI' 'EN' 'SE']
```

Explanation:

- `.unique()` is a Pandas method that returns an array of distinct values in a column.
- It's useful to:
 - Understand what categories exist in your dataset
 - Verify if cleaning (like `.strip()`, `.upper()`, `.title()`) was successful
 - Decide whether any values need to be grouped or reclassified

Handle duplicates:

Syntax:

```
df.drop_duplicates(inplace=True)
```

Explanation:

- **`df.drop_duplicates()` removes duplicate rows** — i.e., rows where **all column values are exactly the same**.
- **`inplace=True` modifies the original DataFrame directly** instead of creating a copy.

Important because:

- Duplicate records can lead to:
 - **Incorrect analysis** (e.g., counting jobs or salaries multiple times)

- **Biased summaries** (e.g., inflated averages, wrong distributions)
- Cleaning them ensures:
 - Accurate data representation
 - Cleaner visualizations and results

Handle duplicates

```
[17]: df.drop_duplicates(inplace=True)
```

Generate Descriptive Statistics:

Explanation:

- `df.describe()` gives summary statistics of your dataset.
- By default, it includes only numeric columns (like salary or duration).
- But with `include='all'`, it shows statistics for all column types — including text (object), categorical, boolean, datetime, etc.

View clean data summary

```
[19]: df.describe(include='all')
```

	job_id	job_title	salary_usd	salary_currency	salary_local	experience_level	employment_type	company_location	company_size
count	15000	15000	15000.000000	15000	1.500000e+04	15000	15000	15000	15000
unique	15000	20	NaN	8	NaN	4	4	20	15000
top	AI00001	Machine Learning Engineer	NaN	USD	NaN	EX	CT	Switzerland	15000
freq	1	824	NaN	7453	NaN	3843	3841	819	508
mean	NaN	NaN	121991.938267	NaN	8.292366e+05	NaN	NaN	NaN	NaN
min	NaN	NaN	16621.000000	NaN	1.662100e+04	NaN	NaN	NaN	NaN
25%	NaN	NaN	74978.500000	NaN	7.383075e+04	NaN	NaN	NaN	NaN

For numeric columns:

- count: number of non-null values
- mean: average
- std: standard deviation
- min: minimum

- 25%, 50%, 75%: quartiles
- max: maximum

For categorical/text columns:

- count: number of non-null entries
- unique: number of unique values
- top: most frequent value
- freq: frequency of the top value

For datetime columns (if any):

- Just count, unique, top, freq, and first, last (in newer Pandas versions)

Outlier Handling:

Outlier Detection and Handling

Q1 and Q3 for salary_usd

Q1 = df['salary_usd'].quantile(0.25)

Q3 = df['salary_usd'].quantile(0.75)

IQR = Q3 - Q1

print(IQR)

Q1 → the value below which 25% of salaries fall.

Q3 → the value below which 75% of salaries fall.

IQR → the spread of the middle 50% of your data (Q3 – Q1).
It tells you how “wide” the middle chunk of the salary distribution is.

lower and upper bounds

lower_bound= Q1 - 1.5 * IQR

upper_bound= Q3 + 1.5 * IQR

print(lower_bound)

print(upper_bound)

This is the **classic box-plot outlier rule**:

- Anything **less than** lower_bound is considered a *low outlier*.
- Anything **greater than** upper_bound is considered a *high outlier*.

The 1.5 is a multiplier chosen by convention—large enough to ignore normal variation, small enough to catch unusual values.

Filtering out outliers

```
outliers_usd = df[(df['salary_usd'] >= lower_bound) & (df['salary_usd'] <= upper_bound)]  
print(outliers_usd)
```

This condition checks each row:

- Keep it if salary_usd is between the lower and upper bound (inclusive).

The result is a DataFrame (outliers_usd) without outliers—really, it’s your clean data.

If you wanted *only* the outliers, you’d flip the condition:

```
Outlier Handling:  
[70]: # Q1 and Q3 for salary_usd  
Q1 = df['salary_usd'].quantile(0.25)  
Q3 = df['salary_usd'].quantile(0.75)  
IQR = Q3 - Q1  
print(IQR)  
  
# Lower and upper bounds  
lower_bound = Q1 - 1.5 * IQR  
upper_bound = Q3 + 1.5 * IQR  
print(lower_bound)  
print(upper_bound)  
  
# Filtering out outliers  
outliers_usd = df[(df['salary_usd'] >= lower_bound) & (df['salary_usd'] <= upper_bound)]  
print(outliers_usd)
```

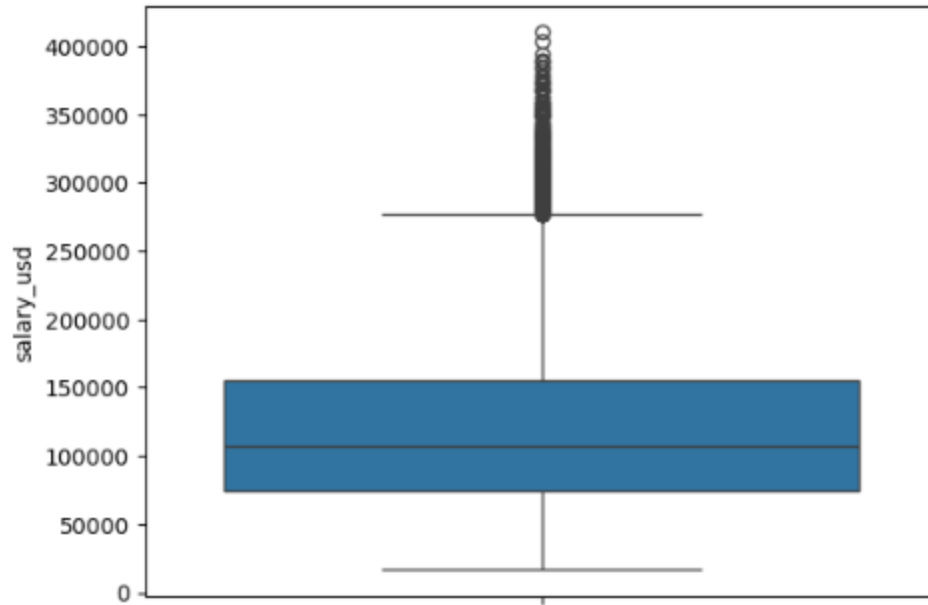
```
80773.75  
-46182.125  
276912.875  
  
   job_id      job_title  salary_usd salary_currency \  
0  AI00001  Data Scientist    219728             USD  
1  AI00002  Head of AI       230237             JPY  
2  AI00003  Data Engineer   128890             EUR  
3  AI00004  Computer Vision Engineer  96349             USD  
4  AI00005  Robotics Engineer   63065             EUR  
...      ...           ...           ...           ...  
14995 AI14996  AI Product Manager   39171             USD  
14996 AI14997  AI Product Manager   77555             EUR  
14997 AI14998  Data Engineer    28380             USD  
14998 AI14999  AI Specialist    58764             EUR  
14999 AI15000  Principal Data Scientist  58623             USD  
  
   salary_local experience_level employment_type company_location \  
0           219728             EX             PT             Sweden
```

```
outliers_usd.shape
```

```
(14591, 21)
```

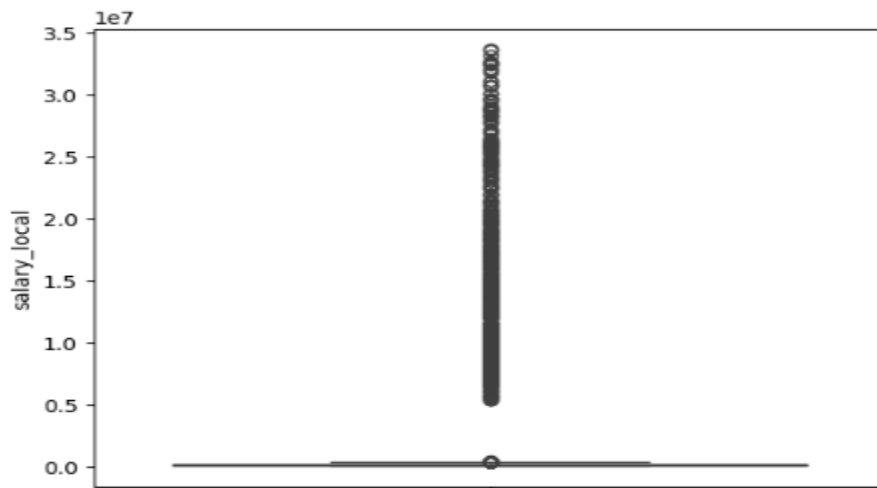
```
import seaborn as sns
import matplotlib.pyplot as plt

sns.boxplot(y=df['salary_usd'])
plt.show()
```



```
sns.boxplot(y=df['salary_local'])
plt.show()
```

```
sns.boxplot(y=df['salary_local'])
plt.show()
```

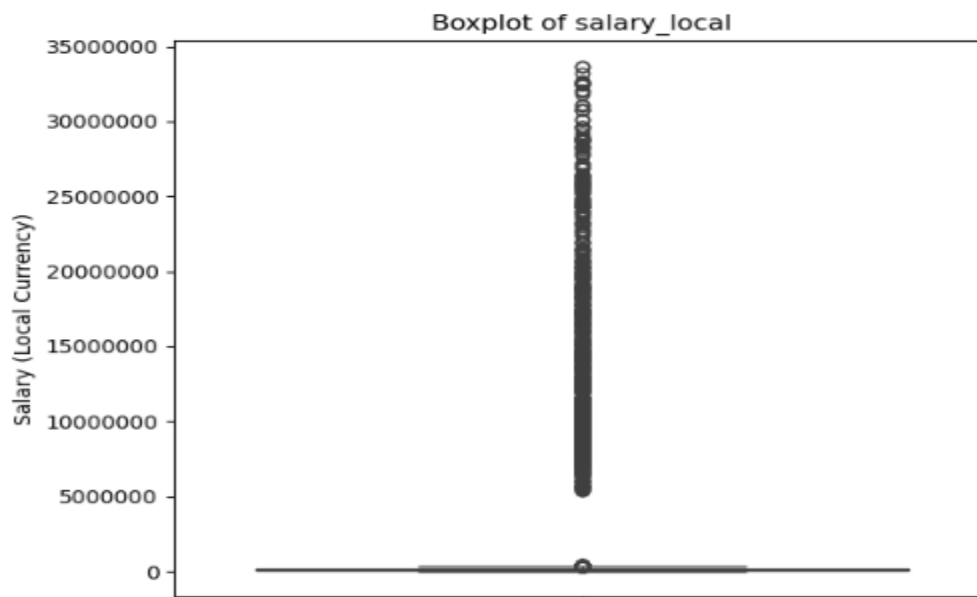


```
import seaborn as sns
import matplotlib.pyplot as plt

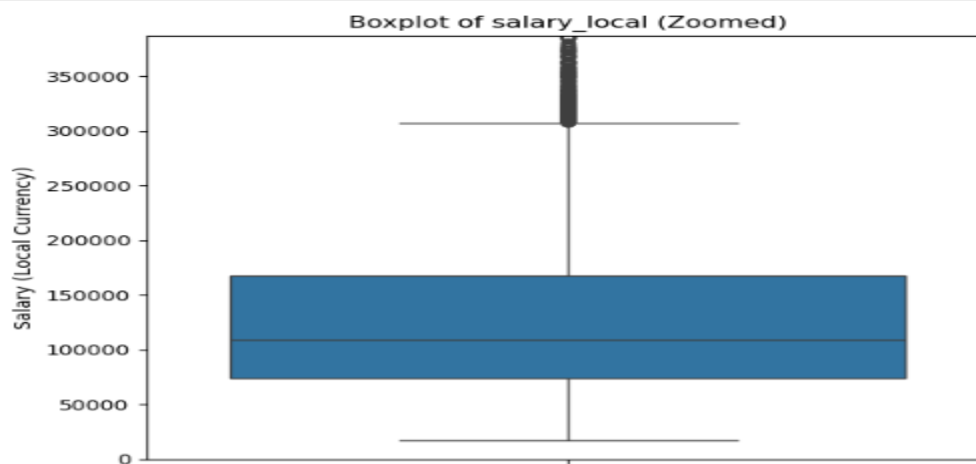
sns.boxplot(y=df['salary_local'])

# Disable scientific notation
plt.ticklabel_format(style='plain', axis='y')

plt.title("Boxplot of salary_local")
plt.ylabel("Salary (Local Currency)")
plt.tight_layout()
plt.show()
```



```
# Zoom in on the middle 95% (without dropping data):
sns.boxplot(y=df['salary_local'])
plt.ylim(0, df['salary_local'].quantile(0.95)) # show up to 95th percentile (lim=Limit)
plt.title("Boxplot of salary_local (Zoomed)")
plt.ylabel("Salary (Local Currency)")
plt.tight_layout()
plt.show()
```



Remove extreme outliers temporarily to analyze the core data:

Remove extreme outliers temporarily to analyze the core data:

```
q1 = df['salary_local'].quantile(0.25)
q3 = df['salary_local'].quantile(0.75)
iqr = q3 - q1
print('Inter Quatrile Range:',iqr)

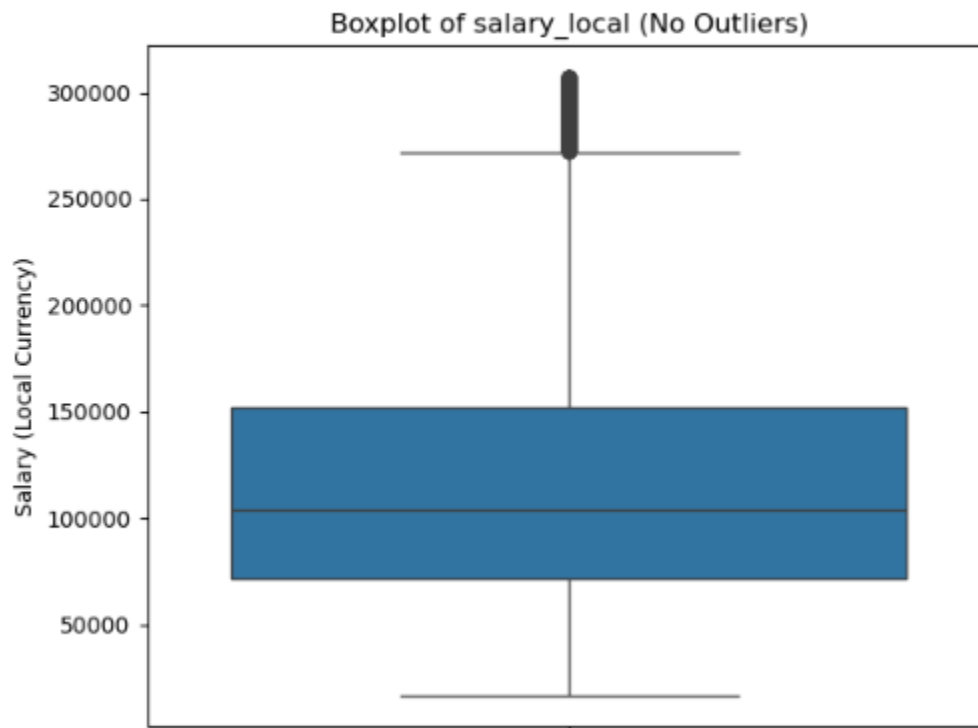
upper = q3 + 1.5 * iqr
lower = q1 - 1.5 * iqr
print('upper boundary:',upper)
print('lower boundary:',lower)

# Filter out outliers
filtered = df[(df['salary_local'] >= lower) & (df['salary_local'] <= upper)]

# Plot without outliers
sns.boxplot(y=filtered['salary_local'])
plt.title("Boxplot of salary_local (No Outliers)")
plt.ylabel("Salary (Local Currency)")
plt.tight_layout()
plt.show()
```

```
Inter Quatrile Range: 93497.0
upper boundary: 307573.25
lower boundary: -66414.75
```

```
Inter Quatrile Range: 93497.0
upper boundary: 307573.25
lower boundary: -66414.75
```



```
query = "SELECT * FROM job_dataset"
df = pd.read_sql(query, engine)
df
```

1	AI00002	Head of AI	230237	JPY	25326070	EX	PT	Japan	L	Japan
2	AI00003	Data Engineer	128890	EUR	109557	EX	CT	Germany	S	Germany
3	AI00004	Computer Vision Engineer	96349	USD	96349	MI	FL	Finland	L	Finland
4	AI00005	Robotics Engineer	63065	EUR	53605	EN	FT	France	S	France
...	...	...	...	...	...	...	...	...	...	...

```
query = "SELECT * FROM job_dataset"
df = pd.read_sql(query, engine)
df
```

	job_id	job_title	salary_usd	salary_currency	salary_local	experience_level	employment_type	company_location	company_size	employee_residence	remote_allowed
0	AI00001	Data Scientist	219728	USD	219728	EX	PT	Sweden	M	Sweden	
1	AI00002	Head of AI	230237	JPY	25326070	EX	PT	Japan	L	Japan	
2	AI00003	Data Engineer	128890	EUR	109557	EX	CT	Germany	S	Germany	
3	AI00004	Computer Vision Engineer	96349	USD	96349	MI	FL	Finland	L	Finland	

```
df_sp = pd.read_sql("CALL analyze_job_market();", engine)
df_sp
```

	job_title	avg_salary_title
0	Data Engineer	127533.77
1	AI Product Manager	126029.68
2	Deep Learning Engineer	124395.49
3	ML Ops Engineer	123466.61
4	NLP Engineer	122867.78
5	Data Scientist	122846.25
6	Machine Learning Researcher	122553.72
7	Machine Learning Engineer	122516.21
8	Research Scientist	122298.95
9	AI Architect	122053.60

```
: import pandas as pd

# Raw DBAPI connection
conn = engine.raw_connection()
cursor = conn.cursor()

# Call the stored procedure
cursor.callproc("analyze_job_market")

# Fetch all result sets manually
dfs = []
while True:
    result = cursor.fetchall()
    if result:
        # Get column names
        columns = [col[0] for col in cursor.description]
        # Create DataFrame
        df = pd.DataFrame(result, columns=columns)
        dfs.append(df)
    if not cursor.nextset():
        break

# Close connection
cursor.close()
conn.close()

# Unpack DataFrames (assuming 6)
(df_title_salary,
 df_country_salary,
 df_jobs_by_company,
 df_jobs_by_industry,
 df_remote_vs_onsite,
 df_exp_level) = dfs
```

```
: print("1. Top 10 Highest-Paying Job Titles:")
print(df_title_salary)

print("\n2. Countries with Highest Average Salary:")
print(df_country_salary)

print("\n3. Number of Jobs Listed by Each Company:")
print(df_jobs_by_company)

print("\n4. Number of Jobs Posted per Industry:")
print(df_jobs_by_industry)

print("\n5. Remote vs On-site Jobs Count:")
print(df_remote_vs_onsite)

print("\n6. Experience Levels with Most Job Postings:")
print(df_exp_level)
```

Output:

1. Top 10 Highest-Paying Job Titles:

	job_title	avg_salary_title
0	Data Engineer	127533.77
1	AI Product Manager	126029.68
2	Deep Learning Engineer	124395.49
3	ML Ops Engineer	123466.61
4	NLP Engineer	122867.78
5	Data Scientist	122846.25
6	Machine Learning Researcher	122553.72
7	Machine Learning Engineer	122516.21
8	Research Scientist	122298.95
9	AI Architect	122053.60

2. Countries with Highest Average Salary:

	company_location	avg_salary_country
0	Switzerland	172995.24
1	Norway	161036.84
2	Denmark	158599.66
3	United States	141987.25
4	Japan	131289.03
5	Netherlands	130924.21
6	United Kingdom	130063.90
7	Singapore	129558.32
8	Australia	125719.94
9	Ireland	124034.30

3. Number of Jobs Listed by Each Company:

	company_name	job_postings
0	Machine Intelligence Group	986
1	DataVision Ltd	963
2	Quantum Computing Inc	958
3	DeepTech Ventures	950
4	Cognitive Computing	948
5	Advanced Robotics	945
6	Autonomous Tech	943
7	Predictive Systems	941
8	TechCorp Inc	940
9	Cloud AI Solutions	930

4. Number of Jobs Posted per Industry:

	industry	total_jobs
0	Government	1035
1	Manufacturing	1035
2	Consulting	1021
3	Finance	1018
4	Automotive	1015
5	Technology	1011
6	Real Estate	1005
7	Gaming	999
8	Telecommunications	995
9	Education	992
10	Energy	989
11	Healthcare	984

12	Retail	978
13	Media	972
14	Transportation	951

5. Remote vs On-site Jobs Count:

	work_type	job_count
0	on_site	4975
1	Hybrid	4991
2	Remote	5034

6. Experience Levels with Most Job Postings:

	experience_level	num_postings
0	EX	3843
1	MI	3764
2	SE	3741
3	EN	3652

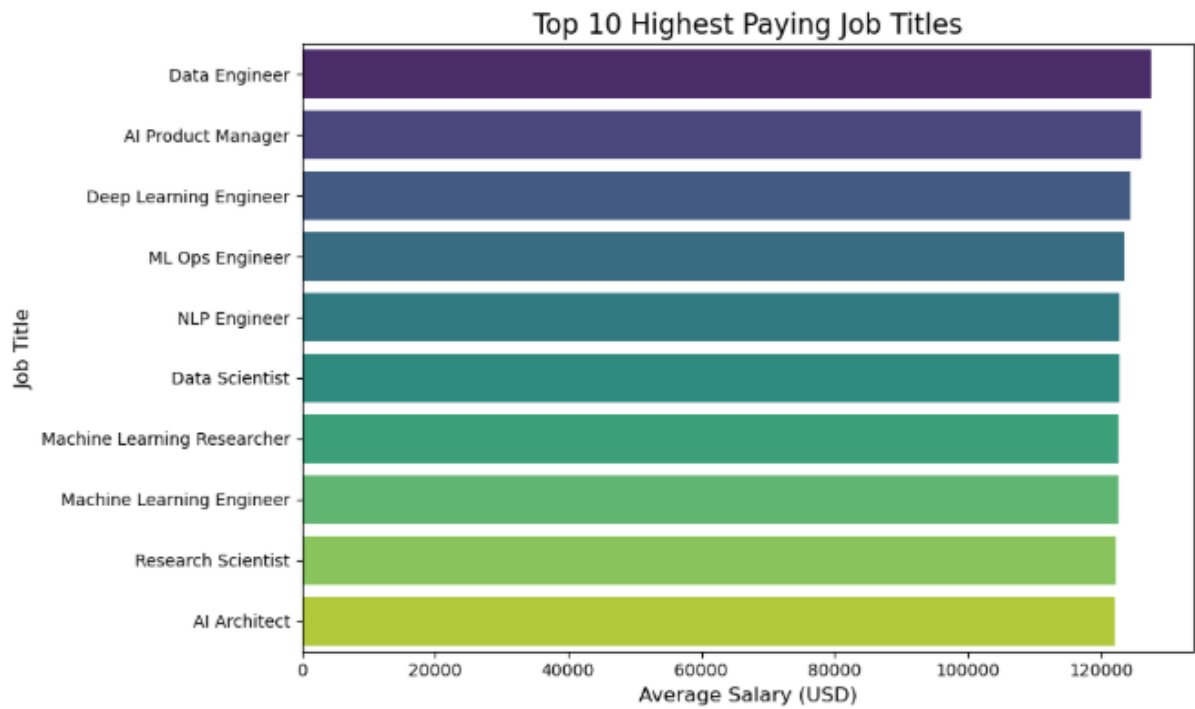
1. Top 10 Highest-Paying Job Titles

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(10, 6))

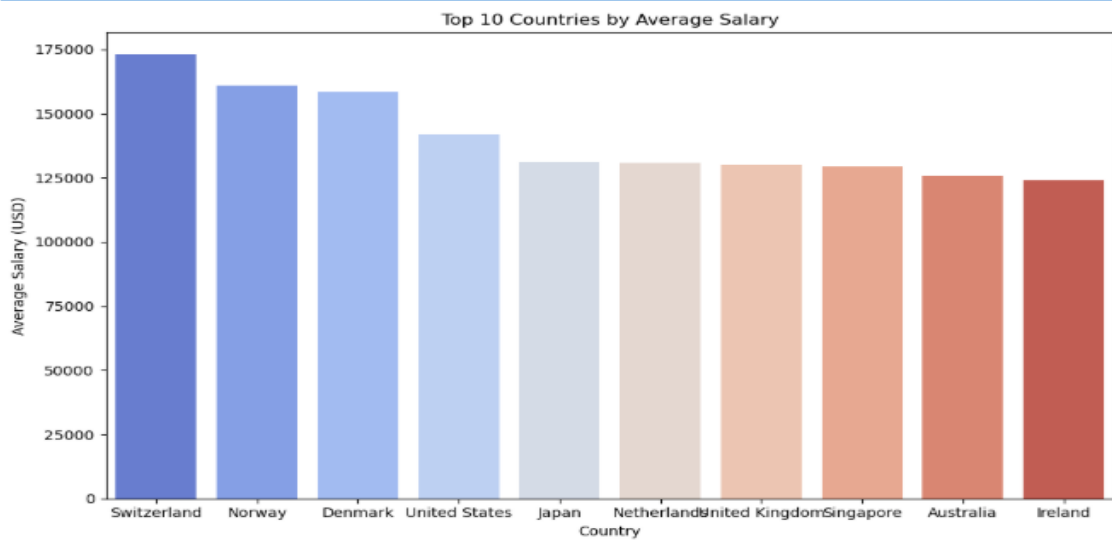
sns.barplot(
    x='avg_salary_title',
    y='job_title',
    data=df_title_salary,
    hue='job_title',          # Set hue to avoid future warning
    palette='viridis',
    legend=False             # Remove redundant Legend
)

plt.title('Top 10 Highest Paying Job Titles', fontsize=16)
plt.xlabel('Average Salary (USD)', fontsize=12)
plt.ylabel('Job Title', fontsize=12)
plt.tight_layout()
plt.show()
```



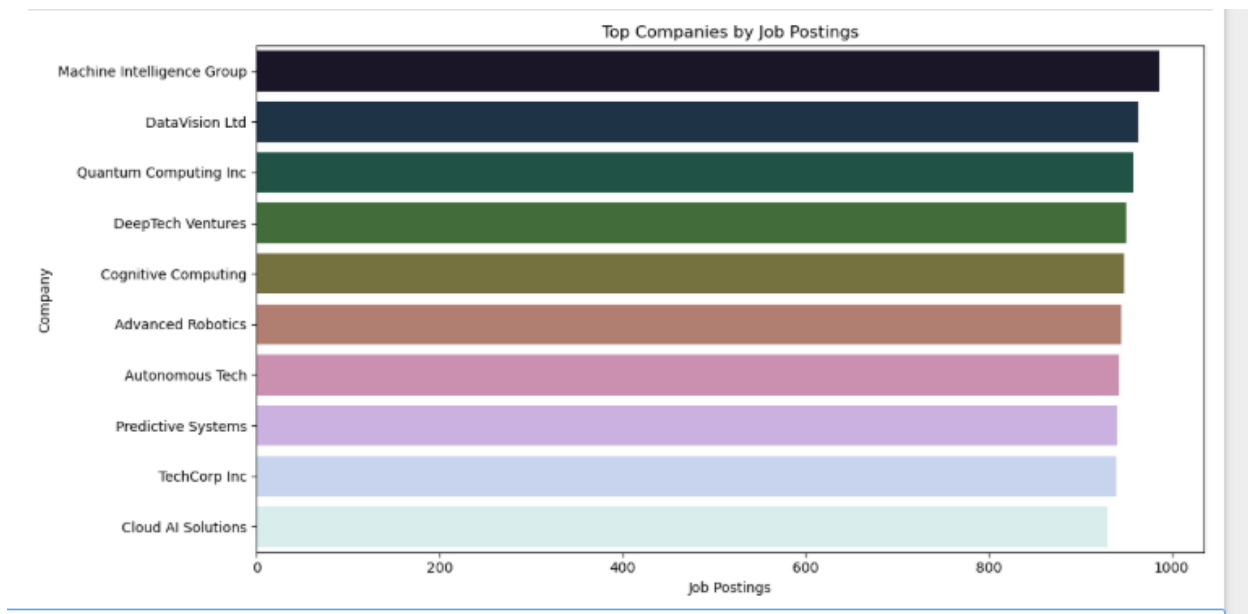
3. Countries with Highest Average Salary

```
plt.figure(figsize=(10, 6))
sns.barplot(x='company_location', y='avg_salary_country', data=df_country_salary, palette='coolwarm')
plt.title('Top 10 Countries by Average Salary')
plt.xlabel('Country')
plt.ylabel('Average Salary (USD)')
plt.tight_layout()
plt.show()
```



3. Number of Jobs Listed by Each Company

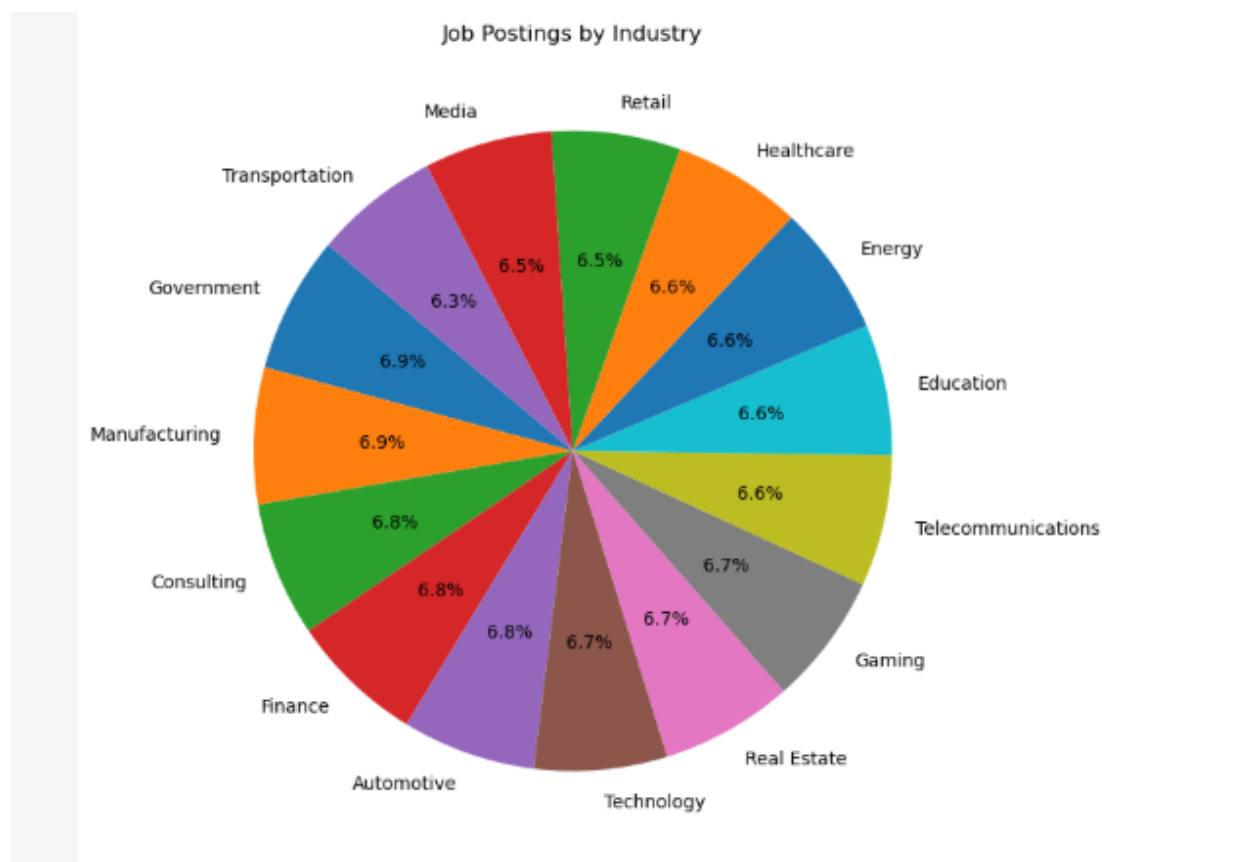
```
plt.figure(figsize=(12, 6))
sns.barplot(x='job_postings', y='company_name', data=df_jobs_by_company, palette='cubehelix')
plt.title('Top Companies by Job Postings')
plt.xlabel('Job Postings')
plt.ylabel('Company')
plt.tight_layout()
plt.show()
```



4. Jobs Posted per Industry

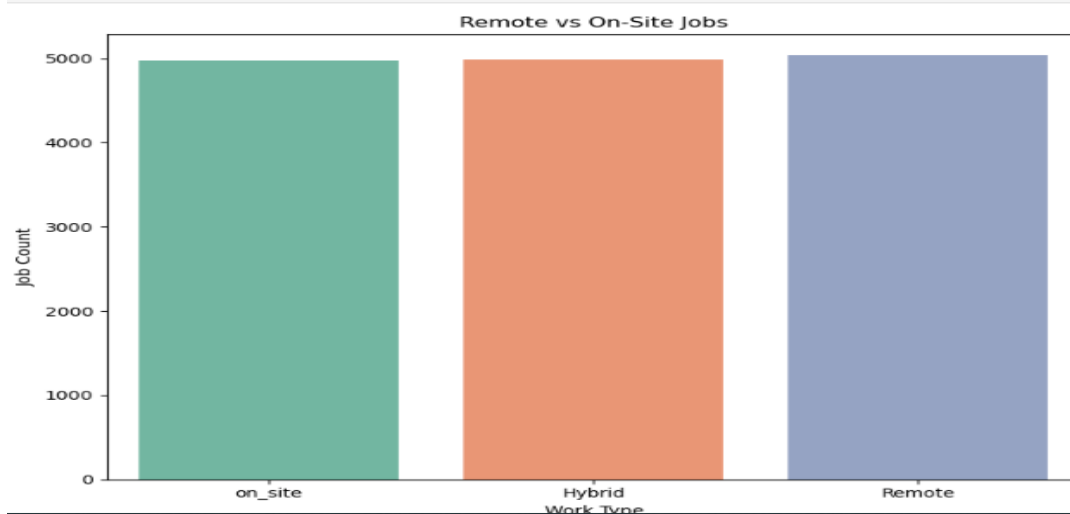
```
query = """
SELECT industry, COUNT(*) AS total_jobs
FROM job_dataset
GROUP BY industry
ORDER BY total_jobs DESC;
"""
df_industry_jobs = pd.read_sql(query, engine)
```

```
plt.figure(figsize=(8, 8))
plt.pie(df_industry_jobs['total_jobs'], labels=df_industry_jobs['industry'], autopct='%1.1f%%', startangle=140)
plt.title('Job Postings by Industry')
plt.tight_layout()
plt.show()
```



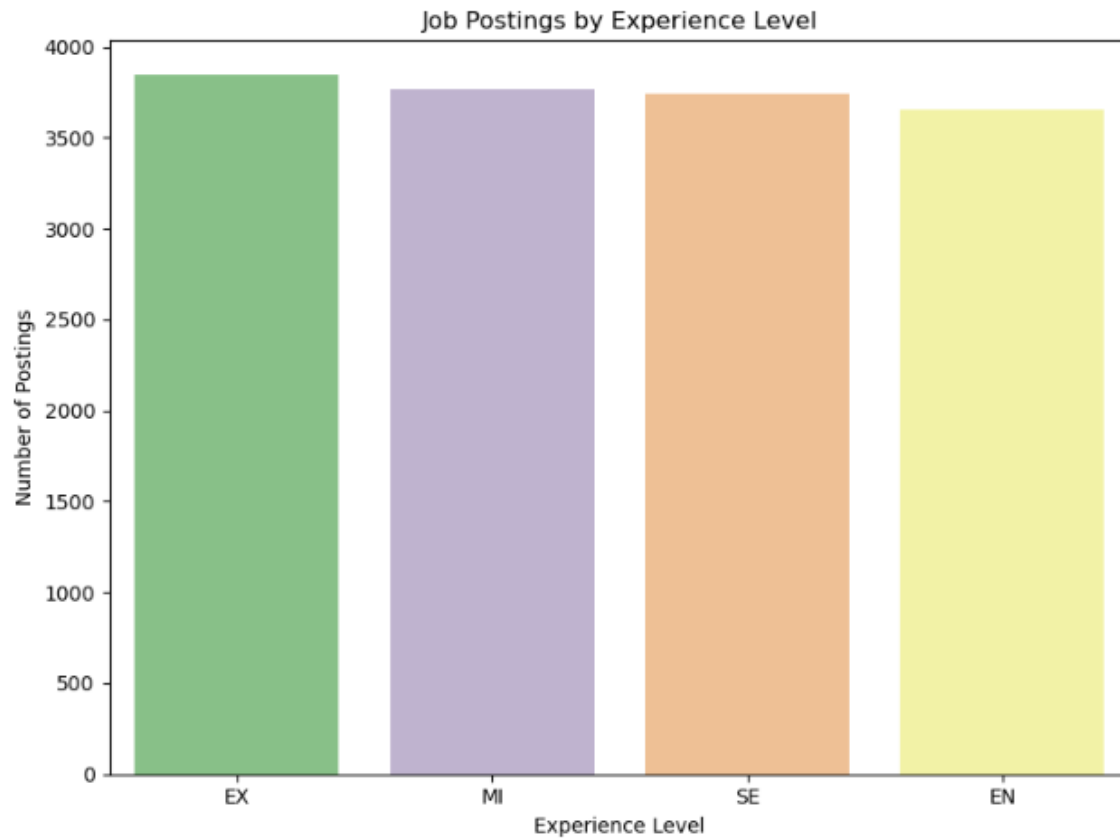
5. Remote vs On-site vs Hybrid Jobs

```
plt.figure(figsize=(8, 6))
sns.barplot(x='work_type', y='job_count', data=df_remote_vs_onsite, palette='Set2')
plt.title('Remote vs On-Site Jobs')
plt.xlabel('Work Type')
plt.ylabel('Job Count')
plt.tight_layout()
plt.show()
```



6. Experience Levels with Most Job Postings

```
plt.figure(figsize=(8, 6))
sns.barplot(x='experience_level', y='num_postings', data=df_exp_level, palette='Accent')
plt.title('Job Postings by Experience Level')
plt.xlabel('Experience Level')
plt.ylabel('Number of Postings')
plt.tight_layout()
plt.show()
```



```
SELECT * FROM job_market.job_dataset;
```

The screenshot shows a database management tool interface. The top bar indicates the database is 'E-Commerce Customer churn db' and the current file is 'SQL File 4*'. The 'job_dataset' table is selected in the 'Schemas' pane on the left. The main query editor displays the SQL statement: `1 • SELECT * FROM job_market.job_dataset;`. The 'Result Grid' shows the first six rows of the table. Below the grid, the 'Output' pane shows the execution log with three messages: 'DEALLOCATE PREPARE stmt' (OK), 'SELECT * FROM job_market.job_dataset LIMIT 0, 2000' (2000 row(s) returned), and 'SELECT * FROM job_market.job_dataset LIMIT 0, 50000' (15000 row(s) returned).

job_id	job_title	salary_usd	salary_currency	salary_local	experience_level	employment_type	company_location	company_size
AI00001	Data Scientist	219728	USD	219728	EX	PT	Sweden	M
AI00002	Head of AI	230237	JPY	25326070	EX	PT	Japan	L
AI00003	Data Engineer	128890	EUR	109557	EX	CT	Germany	S
AI00004	Computer Vision Engineer	96349	USD	96349	MI	FL	Finland	L
AI00005	Robotics Engineer	63065	EUR	53605	EN	FT	France	S
AI00006	Data Scientist	111794	SGD	145332	SE	CT	Singapore	S

This screenshot is a zoomed-in view of the 'Result Grid' and 'Output' panes from the previous image. The 'Result Grid' displays the same six rows of data. The 'Output' pane shows the execution log with three messages: 'DEALLOCATE PREPARE stmt' (OK), 'SELECT * FROM job_market.job_dataset LIMIT 0, 2000' (2000 row(s) returned), and 'SELECT * FROM job_market.job_dataset LIMIT 0, 50000' (15000 row(s) returned).

job_id	job_title	salary_usd	salary_currency	salary_local	experience_level	employment_type	company_location	company_size
AI00001	Data Scientist	219728	USD	219728	EX	PT	Sweden	M
AI00002	Head of AI	230237	JPY	25326070	EX	PT	Japan	L
AI00003	Data Engineer	128890	EUR	109557	EX	CT	Germany	S
AI00004	Computer Vision Engineer	96349	USD	96349	MI	FL	Finland	L
AI00005	Robotics Engineer	63065	EUR	53605	EN	FT	France	S
AI00006	Data Scientist	111794	SGD	145332	SE	CT	Singapore	S

```
use job_market;
```

```
select * from job_dataset;
```

Query Completed

1.What are the top 10 highest-paying job titles?

SELECT job_title,

ROUND(AVG(salary_usd), 2) AS avg_salary_title FROM job_dataset

GROUP BY job_title

ORDER BY avg_salary_title DESC

LIMIT 10;

Query Completed

Data Engineer	127533.77
AI Product Manager	126029.68

Deep Learning Engineer	124395.49
ML Ops Engineer	123466.61
NLP Engineer	122867.78
Data Scientist	122846.25
Machine Learning Researcher	122553.72
Machine Learning Engineer	122516.21
Research Scientist	122298.95
AI Architect	122053.60

2. Which countries have the highest average salary?

```
select company_location, round(avg(salary_usd), 2) as avg_salary_country
from job_dataset group by company_location
order by avg_salary_country desc
limit 10;
```

result:

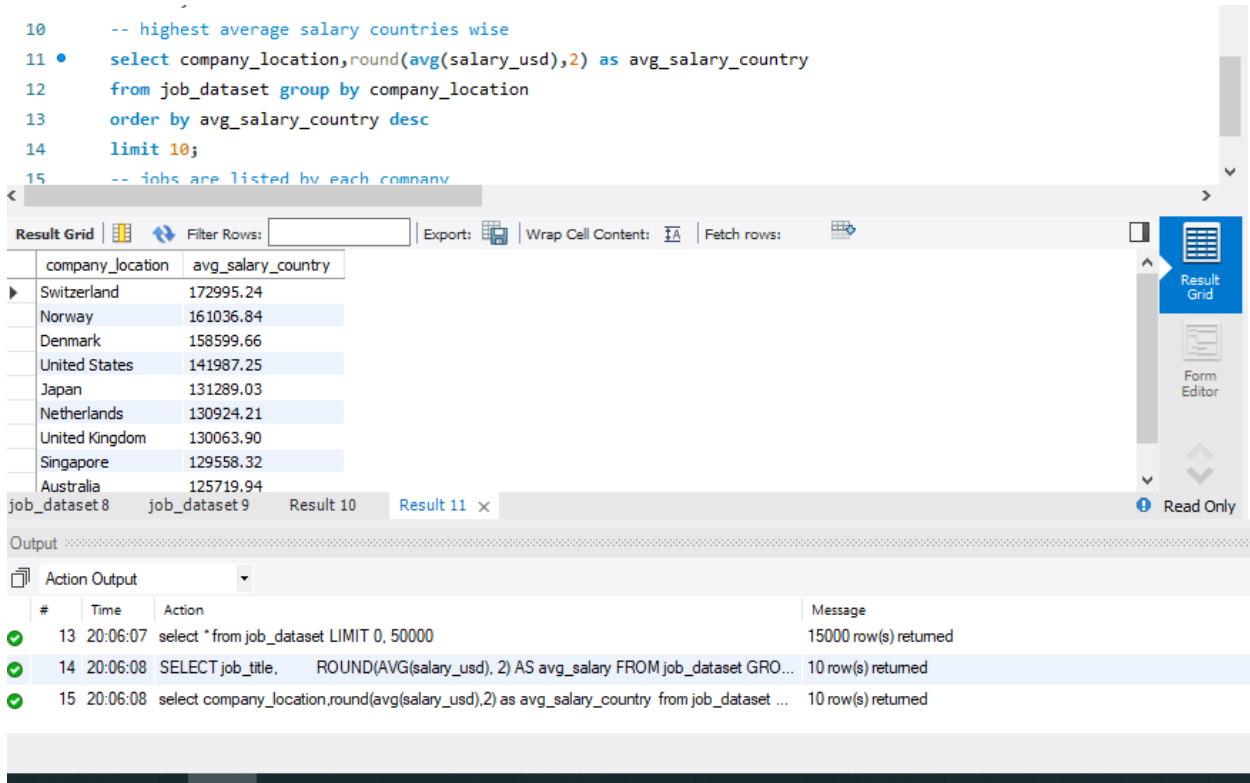
Switzerland	172995.24
Norway	161036.84
Denmark	158599.66
United States	141987.25
Japan	131289.03
Netherlands	130924.21
United Kingdom	130063.90

Singapore 129558.32

Australia 125719.94

Ireland 124034.30

```
10 -- highest average salary countries wise
11 • select company_location,round(avg(salary_usd),2) as avg_salary_country
12 from job_dataset group by company_location
13 order by avg_salary_country desc
14 limit 10;
15 -- jobs are listed by each company
```



company_location	avg_salary_country
Switzerland	172995.24
Norway	161036.84
Denmark	158599.66
United States	141987.25
Japan	131289.03
Netherlands	130924.21
United Kingdom	130063.90
Singapore	129558.32
Australia	125719.94

Output

#	Time	Action	Message
✓ 13	20:06:07	select * from job_dataset LIMIT 0, 50000	15000 row(s) returned
✓ 14	20:06:08	SELECT job_title, ROUND(AVG(salary_usd), 2) AS avg_salary FROM job_dataset GRO...	10 row(s) returned
✓ 15	20:06:08	select company_location,round(avg(salary_usd),2) as avg_salary_country from job_dataset ...	10 row(s) returned

3.How many jobs are listed by each company?

select company_name, count(*) as job_postings from job_dataset

group by company_name

order by job_postings desc

limit 10;

```

15 -- count of jobs are listed by each company
16 • select company_name, count(*) as job_postings from job_dataset
17 group by company_name
18 order by job_postings desc
19 limit 10;
20

```

company_name	job_postings
Machine Intelligence Group	986
DataVision Ltd	963
Quantum Computing Inc	958
DeepTech Ventures	950
Cognitive Computing	948
Advanced Robotics	945
Autonomous Tech	943
Predictive Systems	941
TechCorp Inc	940

Result 19 x

Output

#	Time	Action	Message
✓ 24	20:10:01	select company_location, round(avg(salary_usd), 2) as avg_salary_country from job_dataset ...	10 row(s) returned
✓ 25	20:10:37	SELECT job_title, ROUND(AVG(salary_usd), 2) AS avg_salary_title FROM job_dataset ...	10 row(s) returned
✓ 26	20:32:29	select company_name, count(*) as job_postings from job_dataset group by company_name ...	10 row(s) returned

4. How many jobs are posted per industry?

select industry, count(*) as total_jobs from job_dataset

group by industry order by total_jobs desc;

```

20 -- count of job are posted per industry
21 • select industry, count(*) as total_jobs from job_dataset
22 group by industry order by total_jobs desc;
23

```

industry	total_jobs
Government	1035
Manufacturing	1035
Consulting	1021
Finance	1018
Automotive	1015
Technology	1011
Real Estate	1005
Gaming	999
Telecommunications	995

Result 20 x

Output

#	Time	Action	Message
✓ 25	20:10:37	SELECT job_title, ROUND(AVG(salary_usd), 2) AS avg_salary_title FROM job_dataset ...	10 row(s) returned
✓ 26	20:32:29	select company_name, count(*) as job_postings from job_dataset group by company_name ...	10 row(s) returned
✓ 27	21:05:46	select industry, count(*) as total_jobs from job_dataset group by industry order by total_jobs d...	15 row(s) returned

5. How many remote vs on-site jobs exist?

Let's categorize using remote_ratio column:

- 0 = On-site

- 50 = Hybrid
- 100 = Remote

Syntax:

select

case

when remote_ratio=0 then 'on_site'

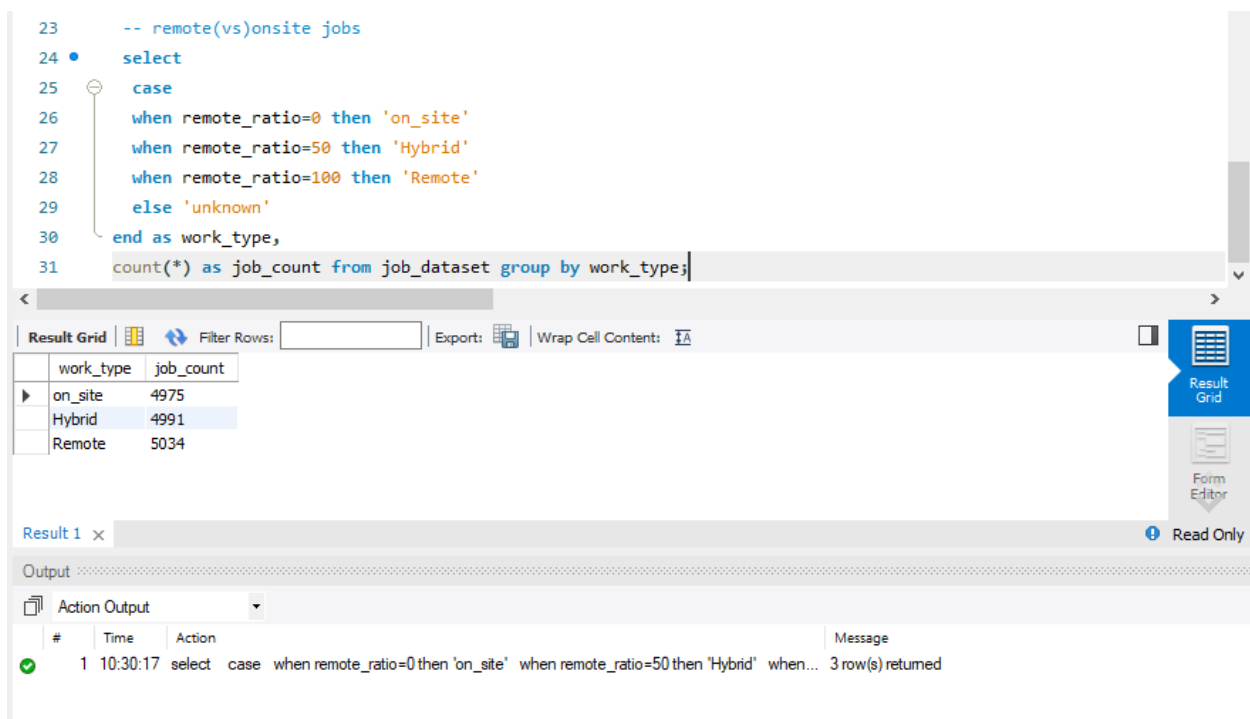
when remote_ratio=50 then 'Hybrid'

when remote_ratio=100 then 'Remote'

else 'unknown'

end as work_type,

count(*) as job_count from job_dataset group by work_type;



The screenshot shows a SQL IDE interface. The top pane displays a SQL query with line numbers 23 to 31. The query is a SELECT statement with a CASE WHEN clause to categorize jobs by remote_ratio into 'on_site', 'Hybrid', 'Remote', or 'unknown', and a COUNT(*) grouped by work_type. The bottom pane shows the 'Result Grid' with 3 rows of data. The 'on_site' row is selected. The 'Output' pane at the bottom shows a message indicating that 3 rows were returned.

```

23  -- remote(vs)onsite jobs
24  •  select
25  case
26  when remote_ratio=0 then 'on_site'
27  when remote_ratio=50 then 'Hybrid'
28  when remote_ratio=100 then 'Remote'
29  else 'unknown'
30  end as work_type,
31  count(*) as job_count from job_dataset group by work_type;

```

work_type	job_count
on_site	4975
Hybrid	4991
Remote	5034

Result 1 x

Output

Action Output

#	Time	Action	Message
1	10:30:17	select case when remote_ratio=0 then 'on_site' when remote_ratio=50 then 'Hybrid' when...	3 row(s) returned

6. Which experience levels have the most job postings?

Syntax:

select experience_level, count(*) as num_postings from job_dataset

group by experience_level

order by num_postings desc;

```
32 -- experience_level most(count) job postings
33 • select experience_level, count(*) as num_postings from job_dataset
34 group by experience_level
35 order by num_postings desc;
```

Result Grid

experience_level	num_postings
EX	3843
MI	3764
SE	3741
EN	3652

Result 2 × Read Only

Output

#	Time	Action	Message
✓ 1	10:30:17	select case when remote_ratio=0 then 'on_site' when remote_ratio=50 then 'Hybrid' when...	3 row(s) returned
✓ 2	10:40:51	select experience_level, count(*) as num_postings from job_dataset group by experience_level...	4 row(s) returned

EN 3652

EX 3843

MI 3764

SE 3741